

Effektivare GET-Endpoints i .NET-

En benchmarkstudie med fokus på svarstider och resursanvändning

Namn	Axel Sköld
Utbildning	.Net Utvecklare
Handledare/ examinator	William Enander
Datum	2025-05-30

Sammanfattning

Denna rapport undersöker tekniker för prestandaoptimering av GET-endpoints i ASP.NET Core API:er. I dagens teknikdrivna samhälle ställs höga krav på systemens svarstider, vilket gör det avgörande för utvecklare att implementera strategier för att bygga effektiva och snabba endpoints. Studien fokuserar på tre optimeringstekniker: `async/await`, `AsNoTracking()` samt in-memory caching med hjälp av `IMemoryCache`. Dessa tekniker förklaras och utvärderas genom tester med verktygen `BenchmarkDotNet` och `Postman CLI`, där svarstider och minnesallokering mäts.

För att genomföra testerna byggdes ett API med ASP.NET Core och Entity Framework Core. Databasen seedades med ett större datamängd genererad via biblioteket `Bogus`, i syfte att efterlikna ett enkelt e-handelssystem. `BenchmarkDotNet` användes för att mäta exekveringstider och minnesanvändning, medan `Postman CLI` simulerade verkliga användare som skickade samtidiga förfrågningar till API:et.

Resultaten visar att in-memory caching med `IMemoryCache` gav den mest betydande prestandavinsten, med svarstider som var flera tusen gånger snabbare än den långsammaste varianten. `AsNoTracking()` visade sig erbjuda en bra balans mellan enkelhet och förbättrad prestanda, medan `async/await` var avgörande för skalbarhet och hantering av samtidiga anrop.

I rapporten dras slutsatsen att en god förståelse för när och hur dessa tekniker bör användas, separat eller i kombination med varandra är avgörande för att bygga snabba, skalbara och pålitliga endpoints.

Abstract

This report investigates performance optimization techniques for GET endpoints in ASP.NET Core APIs. In today's technology-driven society, systems are expected to deliver rapid response times, making it essential for developers to implement strategies that maximize efficiency. The study focuses on three optimization methods: `async/await`, `AsNoTracking()`, and in-memory caching using `IMemoryCache`.

Each technique is described and then evaluated through performance testing using `BenchmarkDotNet` and `Postman CLI` to measure response times and memory allocation. To support the tests, an API was developed using ASP.NET Core and Entity Framework Core, seeded with a large dataset generated via the `Bogus` library to simulate an e-commerce

platform. BenchmarkDotNet was used to analyze low-level execution time and memory usage, while Postman CLI simulated concurrent real-world HTTP requests.

The results indicate that in-memory caching with IMemoryCache produced the most significant performance improvements, with response times several thousand times faster than the slowest alternative. AsNoTracking() offered a strong trade-off between simplicity and performance, and async/await proved crucial for scalability and effective handling of concurrent requests.

The report concludes that understanding when and how to apply each optimization technique, or when to combine them, is vital for developing fast, scalable, and reliable API endpoints.

Lista över förkortningar och begrepp

Begrepp	Term	Beskrivning
Endpoint		En URL eller URI som klienter kan skicka förfrågningar till
Entity Framework		Ett objekt-relationellt mapper verktyg (ORM) som låter en arbeta med databaser via C# klasser istället för SQL.
API	Application Programming Interface	Gränssnitt som låter olika program kommunicera med varandra.
HTTP	Hypertext Transfer Protocol	Kommunikationsprotokoll som definierar hur kommunikation mellan server och klienter går till
GET		HTTP metod som används för att hämta data från en server.
PUT		HTTP metod som används för att uppdatera resurser.
PATCH		HTTP metod som används för att uppdatera delar av resurser.
POST		HTTP metod som används för att skapa data på en server.
DELETE		HTTP metod som används för att ta bort resurser från servern.
SQL	Structured Query Language	Språk för att hantera databaser.
Controller		Klass i .Net som innehåller HTTP metoder.
Seedning		Att fylla databas med data.
Exekvering		När ett program eller en metod körs.
DI	Dependency Injection	En designprincip där beroenden registreras eller skickas in i en komponent.
Trådpool		Samling av trådar som hanterar uppgifter i ett program.
DbContext	Database context	Klass i Entity Framework som representerar sessionen mot databasen.
Caching		Process som lagrar data på serverns minne för bättre prestanda.
Mean		Representerar medelvärde
Allocation		Beskriver fördelning av resurser, i detta fall fördelning av minne.
Key-value par	Key value pair	Två element som hör ihop. Består av en nyckel och ett värde.

Bogus		Ett bibliotek som används för att skapa påhittade data att fylla en databas med.
BenchmarkDotNet		Ett bibliotek för kraftfulla benchmarktester.
Postman CLI		API-klient som används för att testa och dokumentera APIer.
Benchmark		Systematisk mätning av prestanda.
Collection		En samling av API förfrågningar som kan organiseras och körs tillsammans i Postman.
Kallstart (cache)		Första gången en metod med cache körs. Datan är då inte sparad och därför måste hela hämtningen/beräkningen köras.
Latency		Fördröjningen mellan en begäran och svar.
Thread Pool Starvation		När alla tillgängliga trådar är upptagna vilket skapar en kö.

Innehåll

1. Inledning.....	1
1.1 Bakgrund	1
1.2 Syfte	2
1.3 Problemformulering	2
1.4 Avgränsningar och fokus	2
1.5 Metod/Arbetsätt	3
2. Teori	4
2.1 Async/Await.....	4
2.2 AsNoTracking	6
2.3 In-memory Cache	6
2.4 Postman CLI.....	7
2.5 BenchmarkDotNet.....	8
3. Resultat.....	8
3.1 BenchmarkDotNet.....	8
3.2 Postman	8
4. Diskussion	9
4.1 Skillnad mellan testerna	9
4.2 Begränsningar.....	9
4.3 Störst förbättring av prestanda	10
4.4 Skillnad mellan asynkron och synkron	10
4.5 Vilken av teknikerna ger bäst prestanda i förhållande till enkelhet?	10
5. Slutsatser	11
5.1 Rekommendationer	11
6. Referenslista	13
Bilagor	14

1. Inledning

I takt med att dagens system blir allt större och mer invecklade ställs det höga krav på prestanda och optimering. Användare har idag förväntningar på snabbhet, tillgänglighet och sömlösa upplevelser. Detta ställer höga krav på backendsystemen där REST APIer spelar en central roll i kommunikation mellan klient och server. Som utvecklare är det därför extremt viktigt att kunna skriva snabba och pålitliga endpoints för att kunna utveckla skalbara och användarvänliga system. I detta arbete ämnar jag ta reda på hur tre olika optimeringar kan påverka svarstider och resursanvändningar vid hämtning av data. Jag vill utreda vilka av dessa optimeringar för "GET" som är de mest givande utan att för den sakens skull medföra svårsläst och krånglig kod. Optimeringarna jag valt att fokusera på är async/await, AsNoTracking samt caching med IMemoryCache där dessa tekniker kommer att analyseras och utvärderas genom tester.

1.1 Bakgrund

Under min utbildning har jag fått en bred förståelse för hur olika delar inom systemutveckling hänger ihop. På min LIA blev det dock tydligt att det inte räcker med att system bara fungerar, de måste också vara snabba och effektiva, särskilt vid arbete med stora datamängder. Detta skapade en vilja av att lära mig mer om denna avgörande faktor för att bygga säkra och pålitliga applikationer.

REST API:er är centrala i moderna system, och långsamma endpoints kan försämra användarupplevelsen, öka serverbelastningen och medföra högre kostnader. Inom .NET finns flera tekniker för att förbättra prestandan, men det saknas ofta tydlig vägledning kring deras praktiska effekt. Därför vill jag i detta arbete undersöka och jämföra olika optimeringsmetoder för GET-endpoints. för att identifiera vilka som ger mest nytta utan att göra koden onödigt komplex.

1.2 Syfte

Syftet med examensarbetet är att undersöka hur prestandan i GET-endpoints kan påverkas med hjälp av de olika teknikerna `async/await`, `caching` samt `AsNoTracking()`, i syfte att skapa snabbare, mer resurseffektiva och skalbara system.

1.3 Problemformulering

I moderna webbaserade system utgör REST API:er en central del i kommunikationen mellan klient och server. Långsamma endpoints kan försämra användarupplevelsen, leda till ökade kostnader och skapa högre belastning på servrar. Inom .NET finns en rad olika tekniker och metoder för att optimera prestandan i API-endpoints, men informationen kring deras praktiska effekt och eventuella nackdelar i olika situationer är ofta bristfällig.

Detta arbete syftar därför till att:

- Förklara de olika tekniker och strategier för prestandaoptimering av GET-endpoints i .NET som jag valt.
- Jämföra och analysera hur dessa optimeringar påverkar svarstider och resursanvändning.
- Undersöka om vissa optimeringar i vissa sammanhang snarare kan försämra prestanda än förbättra den.

Utifrån dessa punkter har jag kommit fram till fyra frågeställningar som jag vill besvara.

1. Vilken av de tre teknikerna ger mest prestanda?
2. Vilken skillnad är det mellan synkrona och asynkrona anrop i Get-endpoints?
3. Vilken av de testade teknikerna ger störst prestandaförbättring i förhållande till kodens enkelhet?

1.4 Avgränsningar och fokus

I arbetet ämnar jag att endast fokusera på prestandaoptimering i form av ”best practices” för att skriva prestandaeffektiva GET-endpoints. Andra typer av endpoints, såsom POST, PUT, PATCH och DELETE, kommer inte att ingå i denna studie. Även om dessa också är viktiga för helheten i ett API, skiljer sig deras prestandamässiga utmaningar från de som är relevanta vid datahämtning, vilket gör dem mindre intressanta för just denna undersökning. Utöver detta kommer jag inte att inkludera andra designprinciper som rör exempelvis säkerhet eller autentisering. Syftet är istället att rikta fullt fokus på optimering av enskilda GET-endpoints i en traditionell API-struktur. Optimeringarna som används kommer att vara på service-nivå.

Med detta menas att optimeringar på SQL och query nivå inte kommer att inkluderas. Samtliga tester kommer att utföras i en kontrollerad testmiljö för att säkerställa att externa faktorer inte påverkar resultaten. Ingen felhantering kommer heller att användas, då testerna syftar till att mäta ren prestanda under ideala förhållanden, utan att påverkas av logik som hanterar undantag, valideringsfel eller felaktiga indata. Målet är att isolera effekten av respektive optimeringsteknik och undvika att extra kod påverkar mätresultaten.

1.5 Metod/Arbetssätt

I detta arbete ämnar jag att undersöka vilken reell skillnad olika optimeringar av GET-endpoints kan göra när det gäller latens och minnesallokering. För att möjliggöra detta har jag byggt ett testbart API i ASP.NET Core, med en tillhörande SQL Server-databas via Entity Framework Core. Databasen har seedats med hjälp av biblioteket Bogus och innehåller en datamodell som är tänkt att efterlikna en enklare e-handelsplattform.

Totalt består databasen av cirka 15 000 produkter, 2 000 kunder och 10 000 ordrar, där varje order innehåller minst en produktpost. Produkterna kopplas till ordrar genom en separat tabell kallad OrderItems, vilken representerar varje enskild rad i en beställning. Denna tabell innehåller cirka 45 000 rader, vilket ger en lite större volym för att simulera lite tyngre databasoperationer.

I API:ets controller har jag skapat totalt åtta GET-endpoints. Fyra av dessa är enklare metoder som returnerar samtliga produkter i databasen utan någon ytterligare beräkning. De återstående fyra endpoints genomför mer resurskrävande operationer. De sammanställer vilka produkter som sålts flest gånger under de senaste 100 dagarna genom att summera kvantiteter i OrderItems och sedan returnera de tio mest sålda produkterna.

Testerna genomförs dels med hjälp av BenchmarkDotNet för att mäta exekveringstid och minnesförbrukning i isolerade körningar, dels med Postman CLI för att simulera flera simultana användare under last.

För att genomföra mina tester i Postman skapade jag en collection där jag lade in alla åtta GET-endpoints. Sedan använde jag Postman Runner för att köra varje endpoint var för sig i ett eget test. Varje test kördes i två minuter med 100 samtidiga användare som gjorde förfrågningar. Även om det inte helt speglar hur ett riktigt system används, tyckte jag att det var ett rimligt upplägg för att få en bra bild av svarstider och belastning. Fokus låg inte på att efterlikna verkliga användare, utan att mäta hur snabbt mina endpoints svarade under tryck.

Eftersom BenchmarkDotNet egentligen inte är gjort för att testa API-endpoints via HTTP, behövde jag hitta en annan lösning. Jag valde att bygga en BenchmarkService som innehåller anropen till mina metoder. Denna service injicerades via Dependency Injection och anropades sedan direkt inifrån benchmarkklassen. På så sätt kunde jag mäta varje metod i en kontrollerad miljö, utan att påverkas av nätverk eller HTTP. Det gjorde det möjligt att få fram mer exakta siffror för exekveringstid och minnesanvändning, och att jämföra hur de olika optimeringarna faktiskt påverkar prestandan.

2. Teori

Snabba svarstider vid användning av olika tjänster är något vi människor vant oss vid. Ett klick på en hemsida skall ske direkt annars blir vi frustrerade och undrar direkt vad som är fel. För att göra användarupplevelsen smidigare vill man minska svarstiderna. Detta går att göra på många olika sätt, i detta arbete har jag dock valt att fokusera på asynkrona/synkrona anrop, AsNoTracking metoden samt in-memory caching med IMemoryCache. I detta avsnitt kommer varje metod att beskrivas och förklaras.

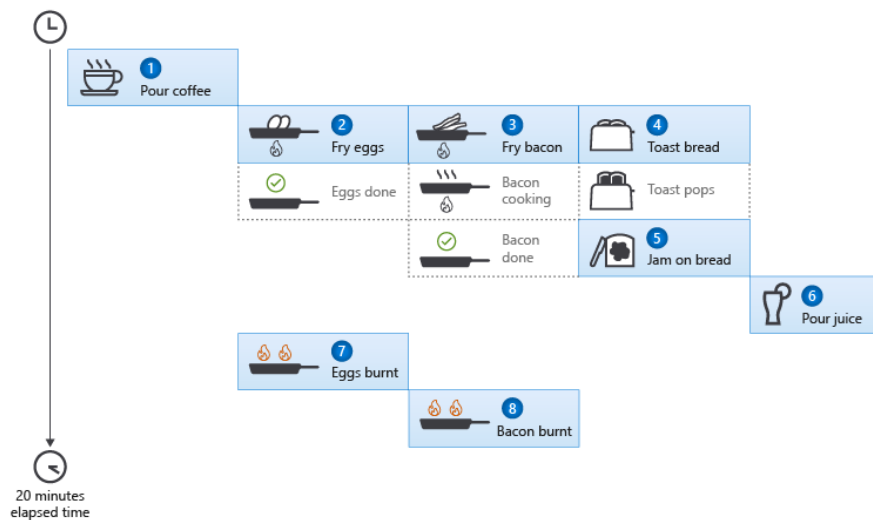
2.1 Async/Await

Async/await i .Net möjliggör att trådar inte blockeras under anrop. När en metod markeras med Async/await frigörs arbetstråden för att kunna hantera andra anrop medan den ursprungliga operationen fortfarande pågår i bakgrunden. Enligt Microsoft([URL1](#)) är ett vanligt prestandafel i appar att man använder sig av synkrona metoder. Synkrona metoder är alltså motsatsen till asynkrona, och blockerar helt tråden till dess att operationen är helt färdig. Detta kan leda till vad man kallar för "Thread Pool Starvation" vilket innebär att antalet tillgängliga trådar är noll och en kö bildas vilket leder till längre svarstider. För APIer innebär Async/await alltså att en trådpool kan hantera flera simultana anrop vilket också är vad Microsoft ([URL1](#)) rekommenderar. Ett bra exempel på hur asynkrona anrop fungerar är att tänka sig att man skall laga en frukost. På (Figur 2.1) kan vi se hur det kan se ut ifall vi lagar frukosten synkront:



Figur 2.1 Exempel på synkron exekvering ([URL2](#))

Det blir inte särskilt effektivt. Som van frukostlagare ser man direkt att det finns flera uppgifter som går att utföra samtidigt, asynkront.



Figur 2.2 Exempel på asynkron exekvering ([URL2](#))

På figuren 2.2 ser vi att den asynkrona varianten är mycket effektivare och vi kan kapa tiden det tar att laga frukosten rätt rejält. Genom att steka ägg, bacon och rosta bröd samtidigt går processen snabbare. Det finns dock en risk, låter vi saker och ting ligga kvar medan vi gör annat så riskerar vi att det blir bränt, vilket illustreras på punkt 7 och 8 i figur 2.2. Med detta menar Microsoft([URL2](#)) att när allt startas samtidigt, men inget händer direkt när något är klart, kan resultatet förstöras – som att baconet bränns medan man gör något annat. Exempelvis stödjer inte Entity Framework parallella databasoperationer på samma DbContext vilket gör det extra viktigt att använda sig av nyckelordet `await` ([URL2](#)).

2.2 AsNoTracking

Entity Framework Core håller automatiskt koll på alla objekt som hämtas från databasen, så att det kan upptäcka om något ändras och spara dessa ändringar när man använder `SaveChanges()` ([URL3](#)). Detta innebär att alla objekt som Entity Framework laddar in från databasen inte bara hämtas och returneras till klienten, utan också sparas i minnet. Detta har många fördelar men kan också kosta en hel del resurser då minne måste allokeras. Detta innebär att vid scenarion där man endast är ute efter att läsa data, kan det vara mer fördelaktigt att använda sig av `query`s utan tracking ([URL3](#)). Enligt en undersökning gjord av Codingbolt i EF Core 7 så sjönk exekveringstiden från 31.1 ms i snitt till 6.15 ms ([URL4](#)) med `AsNoTracking()`. Viktigt är dock att fortsatt ha med sig att `query`s utan tracking endast lämpar sig för rena läsfrågor. `AsNoTracking` är enkel att använda då den bara behöver läggas till i slutet av en `query` för inaktivera spårningen.

2.3 In-memory Cache

Caching är en kraftfull teknik för att förbättra prestanda i webbtjänster. Genom att spara redan hämtade eller beräknade data i minnet slipper applikationen göra om samma arbete vid varje förfrågan. Det här är särskilt effektivt vid data som sällan ändras, men som används ofta – till exempel topplistor, inställningar eller innehåll som hämtas från externa källor.

I ASP.NET Core finns flera olika sätt att implementera cache. Den enklaste och mest använda metoden är `IMemoryCache`, som sparar data i serverns RAM-minne ([URL5](#)). Cachen är intern för webbapplikationen och hanterar all data som `key-value`-par, vilket gör den snabb att slå upp. När ett värde väl har lagrats i cachen, kan det vid nästa anrop returneras direkt – utan att exempelvis behöva gå till databasen igen. Det minskar svarstiden drastiskt, ibland till bara några nanosekunder.

Enligt Microsoft([URL5](#)) kan alla typer av objekt sparas i IMemoryCache, så länge man kan ge dem en unik nyckel. Det gör cachen flexibel och enkel att använda i API-endpoints som ofta returnerar samma typ av data.

För att undvika att datan blir föråldrad eller att minnet fylls upp, är det viktigt att styra hur länge ett cache-objekt skall leva. IMemoryCache erbjuder två huvudsakliga sätt att styra detta:

- Absolute expiration: anger en fast tidpunkt eller ett tidsintervall efter vilket objektet automatiskt tas bort från cachen.
- Sliding expiration: innebär att ett objekt tas bort om det inte har använts under en viss tid. Tiden räknas om varje gång objektet används.

Båda dessa sätt hjälper till att säkerställa att cachedata hålls uppdaterad och att onödiga objekt inte ligger kvar i minnet längre än nödvändigt. Det går även att sätta prioritet på cacheobjekt, vilket gör att objekt med låg prioritet tas bort först om minnet börjar ta slut ([URL5](#)).

Det är också viktigt att förstå att IMemoryCache inte är delad mellan flera servrar. I ett system med flera instanser av en webbapplikation (till exempel i en serverfarm eller molntjänst) har varje instans sin egen separata cache. För sådana scenarier bör man istället använda en distribuerad cache, som till exempel Redis ([URL5](#)).

Första gången en endpoint anropas så är cachen tom, och all data måste hämtas eller beräknas från grunden. Detta kallas för kallstart. När cachen är full så kallas detta för varmstart.

Sammanfattningsvis är IMemoryCache en enkel och effektiv lösning för att snabba upp svarstider och minska belastningen på databasen. Genom att konfigurera utgångstider och hålla koll på cachevolymen går det att få mycket prestanda för liten insats.

2.4 Postman CLI

Postman är en API plattform som hjälper en att bygga, testa och arbeta med APIer ([URL6](#)). I Postman kan Collections skapas, där man genom Postmans inbyggda Collection Runner kan testa endpoints på olika sätt. Bland annat finns det en del för att testa prestanda där man kan ställa in antalet simulerade användare samt tid för testet. Man kan också välja en "load profile" som styr ifall antalet simulerade användare skall ändras under testets gång. I detta arbete används profilen "fixed" som innebär att antalet användare är konstant.

2.5 BenchmarkDotNet

BenchmarkDotNet är ett benchmarkingramverk för som är utvecklat för att testa och analysera exekveringstid och minnesallokering. Varje test körs många gånger för att ge ett stabilt mätvärde ([URL7](#)). I testerna i detta arbete har varje endpoint kapslats in i egen metod och körs via Dependency Injection, då BenchmarkDotNet inte stödjer rena HTTP förfrågningar ([URL8](#)).

3. Resultat

I alla tester mäts svarstider i MS (millisekunder) och minnesallokering i B (Byte).

3.1 BenchmarkDotNet

I dessa tester fokuserar vi på "Mean" vilket innebär medelvärdet för varje endpoints svarstid samt "Allocation" vilket är medelvärde för minnet som allokerats vid körning av varje endpoint. Minnet mäts i byte.

Method	Mean (ms)	Allocated (B)
GetProductsNoAsync	6.4342104	3987493
GetProductsWithAsync	13.2323491	80781404
GetProductsAsNoTracking	14.8478089	206824683
GetProductsCached	0.0001165	344
GetTopProductsNoAsync	39.9961641	29655
GetTopProductsWithAsync	41.8912561	33809
GetTopProductsAsNoTracking	42.1057388	35329
GetTopProductsCached	0.0001137	368

Figur 3.1 BenchmarkDotNet Testresultat

Testet visar tydligt att metoderna med cache var överlägset snabbast och allokerade minst minne. Vi kan också se att de rent synkrona varianterna är snabbare och allokerar mindre minne än de asynkrona. Båda varianter av AsNoTracking() är också både de långsammaste samt de som allokerar mest minne.

3.2 Postman

I Postman testerna tittar vi på medelvärde för svarstid (Mean) samt P90 vilket representerar svarstiden för 90% av de snabbaste svarstiderna. Detta utgör en god markör för svarstiden för de flesta av de simulerade användarna.

Method	Mean (ms)	P90 (ms)
GetProductsNoAsync	4650	4343
GetProductsWithAsync	3292	4081
GetProductsAsNoTracking	2216	3083
GetProductsCached	477	1548
GetTopProductsNoAsync	1351	2120
GetTopProductsWithAsync	1155	1842
GetTopProductsAsNoTracking	981	1827
GetTopProductsCached	89	110

Figur 3.2 Postman Testresultat

Testet visar här att den synkrona varianten av både endpointsen var långsammare än den asynkrona motparten. AsNoTracking() var snabbare än både den synkrona och asynkrona. Även i detta test så är den cacheade varianten helt överlägsen, även fast den i detta test fick två kallstarter. Vi kan också se att P90 resultaten är betydligt högre än medelvärdet, vilket tyder på att det finns anrop som tog betydligt längre tid än andra. Exempelvis så har ”GetProductsCached” en hög P90, kanske på grund av den extra kalla starten som drar upp värdet.

4. Diskussion

4.1 Skillnad mellan testerna

Det är tydligt att det finns skillnader mellan de två olika testernas svar. Till exempel kan vi se på figur 3.1 att medelvärdena för responstiden är mycket lägre än i figur 3.2. Detta då BenchmarkDotNet mäter en enskild metod helt isolerat utan att några HTTP anrop sker. Dessa anrop påverkas inte av några externa faktorer som exempelvis nätverkslatens. Postman simulerar i stället riktiga användare som gör simultana anrop på endpointen. Detta ger en lite mer verklighetstrogen data och denna skillnad kan förklara varför de asynkrona metoderna ser långsammare ut än de synkrona i BenchmarkDotNet testerna.

4.2 Begränsningar

Det finns begränsningar i testupplägget som kan påverka hur applicerbara testresultaten blir i verkliga scenarion. Även om testerna med Postman simulerar verkliga användare så speglar det inte verkliga användarmönster eller latens i form av olika nätverk med varierande hastigheter. I BenchmarkDotNet mäts allting lokalt vilket då helt exkluderar externa faktorer. Då hela testmiljön också kördes lokalt så kan resultaten skilja sig mot om man hade kört i en molnmiljö. Viktigt att nämna är också att endpointen som returnerar produkter inte är särskilt

verklighetstrogen. I riktiga scenarion hade man paginerat produkterna istället för att returnera en lista med alla produkter.

4.3 Störst förbättring av prestanda

Resultaten från både BenchmarkDotNet och Postman testerna visar enhetligt att Microsofts uppmaning till att aggressivt cachea sina endpoints är mycket rimlig([URL1](#)). De cacheade versionerna av endpointsen uppvisar svarstider som är mycket lägre än alla de andra optimeringarna. Även med två kalla starter var det tveklöst att IMemoryCache var allra snabbast. Exempelvis kan vi se i figur 3.1 att den cacheade varianten av "GetTopProducts" var flera tusen gånger snabbare än den långsammaste: AsNoTracking(). I figur 3.2 kan vi dock se att AsNoTracking() snabbare än både asynkron och synkron gällande båda varianterna av endpoints. När det kommer till de asynkrona anropen så är dessa långsammare i BenchmarkDotNet men snabbare i Postman.

4.4 Skillnad mellan asynkron och synkron

Den största skillnaden mellan de synkrona och asynkrona anropen blev tydliga i hur de hanterade belastning med många simulerade användare i Postman än ren exekveringstid i BenchmarkDotNet.

Anledningen till detta är som tidigare nämnt att frigörelsen av trådar med asynk först ger resultat när en metod anropas samtidigt flera gånger. I Postman blir trådarna låsta i den synkrona varianten och resultatet blir därför att den genomsnittliga svarstiden går upp. Detta visar att async/await är att föredra i ett verkligt scenario där skalbarhet och simultana förfrågningar spelar in. Däremot kan vi se att i ren benchmark så är de synkrona anropen snabbare när det gäller att returnera både rena data och den lite tyngre endpointen med topp 10 sålda produkter. Sammanfattningsvis så går det att argumentera för att skillnaden mellan asynkrona och synkrona anrop i detta fall inte var så stor, det går dock att se att ifall man skalar upp antalet användare så kommer den asynkrona varianten att segra i längden.

4.5 Vilken av teknikerna ger bäst prestanda i förhållande till enkelhet?

Alla tekniker som använts i detta arbete kräver några större kodmängder och är relativt enkla att implementera. AsNoTracking kräver endast ett metodanrop i slutet av queryn och kan enkelt läggas till i metoder där den passar in, som exempelvis i endpoints som bara returnerar konsekvent data.

Async/await används numera som standard i .NET och är något Microsoft rekommenderar. Även om det inte alltid är snabbare i tester där man kör en metod i taget, visar det sitt värde när många användare anropar API:et samtidigt. Det största argumentet för denna optimering är därför skalbarheten och att den är snabbare i verkliga scenarion.

Caching med IMemoryCache är den optimering som kräver mest arbete vid implementation. Cachen måste konfigureras med nycklar och utgångstider. Vinsterna rent prestandamässigt är dock enorma och det är svårt att argumentera för något annat än att detta är den bästa optimeringen. Samtidigt så måste ett slag slås för AsNoTracking(). Det är en otroligt enkel lösning som i mindre system kan göra en reell skillnad.

5. Slutsatser

Syftet med detta arbete var att utröna skillnader mellan tre olika optimeringstekniker och vilken skillnad dessa kan göra på GET-endpoints i ASP.Net Core. Genom att analysera testresultaten kan vi tydligt se att caching med IMemoryCache är den överlägset snabbaste och i ett verkligt scenario något man verkligen borde använda där det är lämpligt. En viktig fråga i arbetet var vilken optimering som ger bäst prestanda i förhållande till enkelhet och där blir det tydligt att AsNoTracking() vinner. För ett mindre API där hämtning av konstant data så får den ses som en effektiv lösning. När det gäller de asynkrona anropen så var dessa inte alltid snabbare i ren prestanda men det visar sig tydligt att det är samtidigtheten och skalbarheten av dessa som gör skillnaden i längden.

Sammanfattningsvis så är IMemoryCache överlägset bäst prestandamässigt, AsNoTracking() ger mycket prestanda i kontrast till sin enkelhet och async/await är extremt viktig för skalbarhet. Tanken med att använda både Postman och BenchmarkDotNet anser jag visade sig klokt då de tillsammans kan ge en bättre bild av hur verkligheten ser ut. Att bara köra endpoints i en ren benchmark kan självklart säga något om prestanda, framför allt minnesallokeringen. Dock blir det tydligt att samtidigthet för ett API antagligen är viktigare när det kommer till responstid än ren exekveringstid.

5.1 Rekommendationer

När det kommer vill vilka optimeringar man skall välja för sitt API så behöver det inte vara så svart och vitt som det målas upp i detta arbete. Dessa optimeringar kan också arbeta tillsammans för att generera ännu snabbare resultat. Viktigt att komma ihåg är också att när

man pratar om tid i dessa sammanhang så talar man om millisekunder. En sekund är 1000 millisekunder, vilket betyder att resultaten från testerna visar exekveringstider från några sekunder till några hundradelar av en sekund. Det hade varit intressant att skala upp hämtningen av data ännu mer för att på så sätt se ännu tydligare skillnader. Framförallt de asynkrona mot de synkrona metoderna där jag tror att de asynkrona hade varit avsevärt snabbare när de synkrona börjar låsa trådar på allvar.

Med detta sagt så följer här de rekommendationer jag kan ge för att optimera prestandan i ett API:

- Följ Microsofts uppmaning till använda caching. Det ger överlägset snabbast respons och upptar allra minst minne, framför allt vid tyngre och återkommande beräkningar.
- Använd `AsNoTracking()` vid rena läsoperationer där data inte ändras.
- Använd alltid `async/await`. Även om resultatet visade att synkrona metoder kan vara snabbare vid läsoperationer så ger samtidigtheten och skalbarheten tunga fördelar.

6. Referenslista

Internet

URL1: Microsoft. (2024). *ASP.NET Core Best Practices*.

<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/best-practices?view=aspnetcore-9.0>

URL2: Microsoft. (2025). *Asynchronous Programming*.

<https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/>

URL3: Microsoft. (2023). *Tracking vs. No-Tracking Queries*.

<https://learn.microsoft.com/en-us/ef/core/querying/tracking>

URL4: Boltsis, E. (2023). *AsNoTracking() Performance in EF Core*.

<https://codingbolt.net/2023/08/23/asnotracking-performance-in-ef-core/>

URL5: Microsoft. (2024). *Cache in-memory in ASP.Net Core*.

<https://learn.microsoft.com/en-us/aspnet/core/performance/caching/memory?view=aspnetcore-9.0>

URL6: Postman. (2025). *What is Postman?*.

<https://www.postman.com/product/what-is-postman/>

URL7: BenchmarkDotNet. (2024). *BenchmarkDotNet – Powerful .Net library for benchmarking*.

<https://benchmarkdotnet.org/>

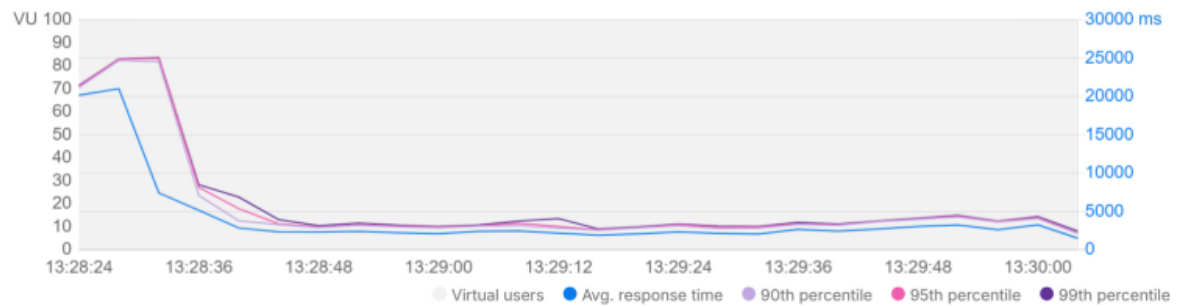
URL8: Chaitanya Penmetsa. (2024). *BENCHMARKING Rest API using Benchmark .NET in .NET 8*.

<https://medium.com/codenx/benchmarking-rest-api-using-benchmark-net-in-net-8-ba7701aa4bcb>

Bilagor

1.1 Response time

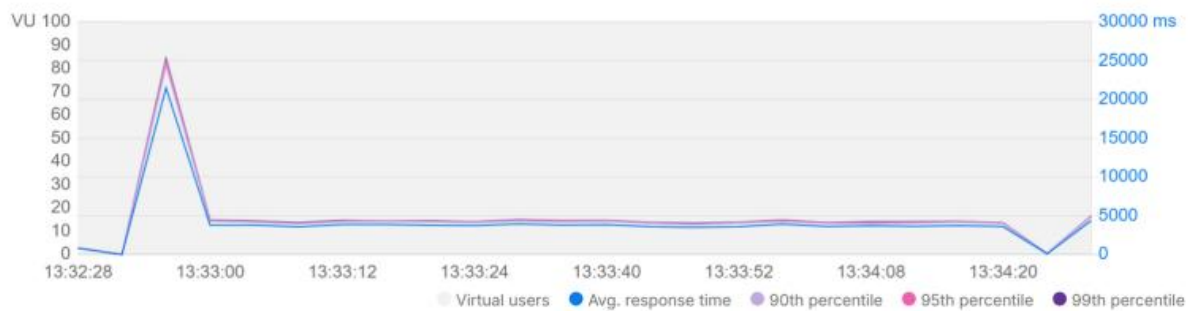
Response time trends during the test duration.



Postman test "GETPRODUCTSWITHASYNC".

1.1 Response time

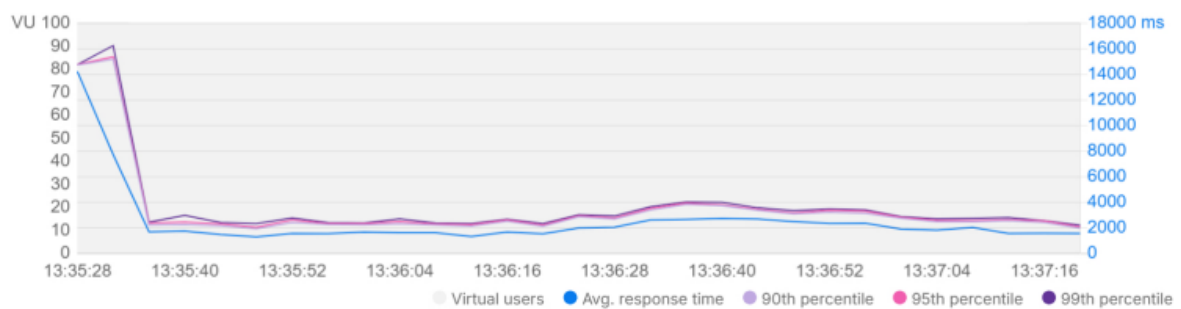
Response time trends during the test duration.



Postman test "GETPRODUCTSNOASYNC".

1.1 Response time

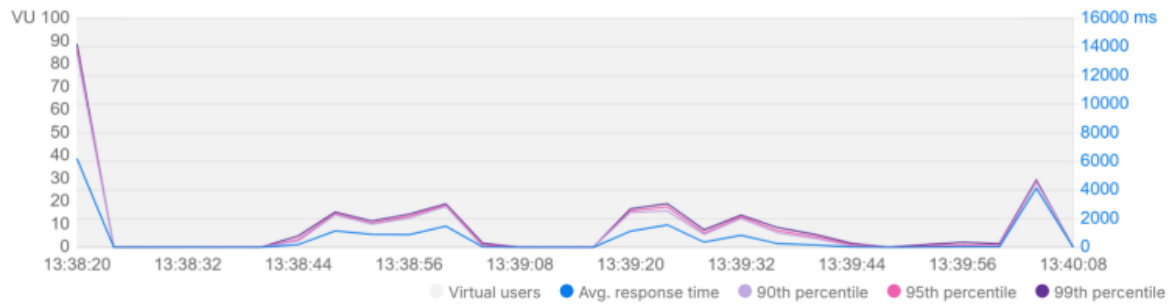
Response time trends during the test duration.



Postman test "GETPRODUCTSASNOTRACKING".

1.1 Response time

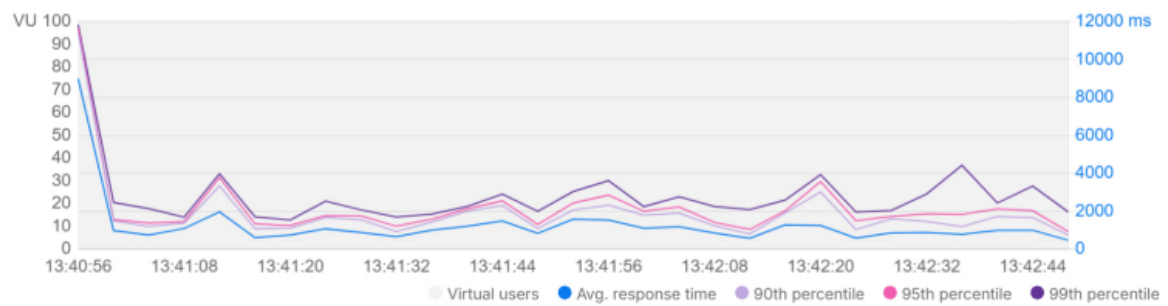
Response time trends during the test duration.



Postman test "GETPRODUCTSCACHED".

1.1 Response time

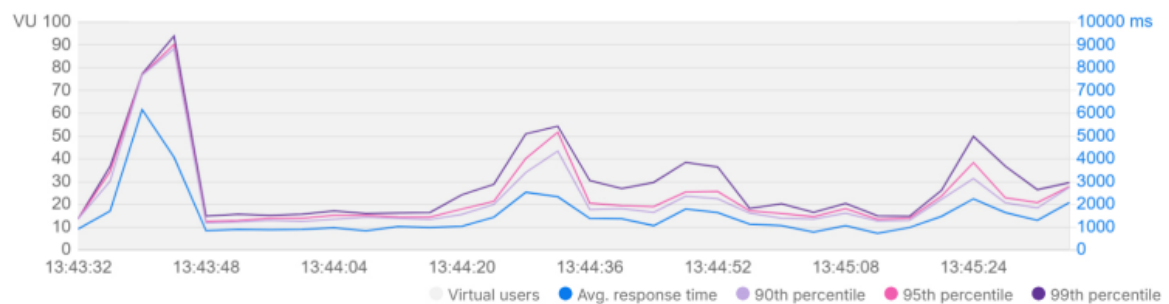
Response time trends during the test duration.



Postman test "GETTOPPRODUCTSWITHASYNC".

1.1 Response time

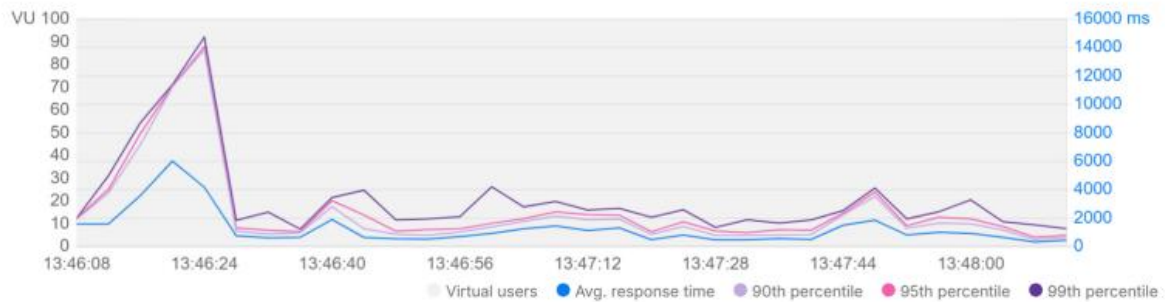
Response time trends during the test duration.



Postman test "GETTOPPRODUCTSNOASYNC".

1.1 Response time

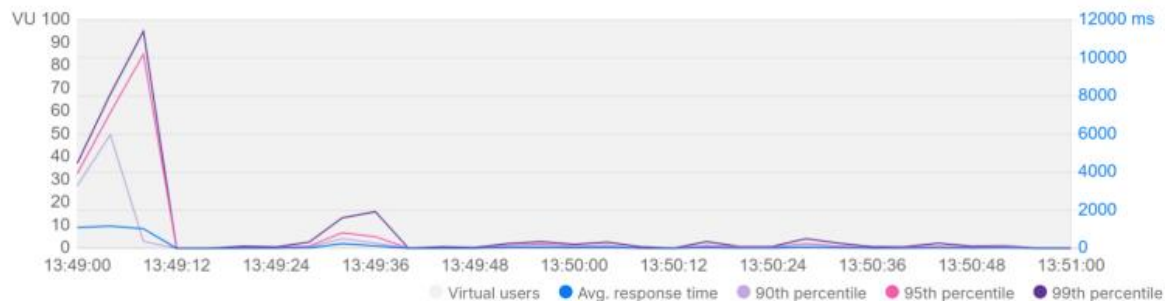
Response time trends during the test duration.



Postman test "GETTOPPRODUCTSASNOTRACKING".

1.1 Response time

Response time trends during the test duration.



Postman test "GETTOPPRODUCTSCACHED".

```

[HttpGet("asnotracking")]
1 reference
public async Task<ActionResult<IEnumerable<Product>>> GetProductsAsNoTracking()
{
    var products = await _context.Products.AsNoTracking().ToListAsync();
    return products;
}

[HttpGet("sync")]
1 reference
public ActionResult<IEnumerable<Product>> GetProductsSync()
{
    return _context.Products.ToList();
}

[HttpGet("async")]
1 reference
public async Task<ActionResult<IEnumerable<Product>>> GetProductsAsync()
{
    return await _context.Products.ToListAsync();
}

[HttpGet("cached")]
1 reference
public async Task<ActionResult<IEnumerable<Product>>> GetProductsCached()
{
    if (_cache.TryGetValue<IEnumerable<Product>>(ProductsCacheKey, out var cachedProducts))
    {
        return Ok(cachedProducts);
    }
    var products = await _context.Products.AsNoTracking().ToListAsync();
    var cacheOptions = new MemoryCacheEntryOptions()
        .SetAbsoluteExpiration(TimeSpan.FromSeconds(60))
        .SetSlidingExpiration(TimeSpan.FromSeconds(20));

    _cache.Set(ProductsCacheKey, products, cacheOptions);

    return Ok(products);
}

```

Metoder som hämtar alla produkter.

```

[HttpGet("top-sync")]
1 reference
public ActionResult<IEnumerable<ProductSalesDto>> GetTopProductsSync()
{
    var since = DateTime.UtcNow.AddDays(-DaysWindow);

    var data = _context.OrderItems
        .Where(oi => oi.Order.OrderDate >= since)
        .GroupBy(oi => new { oi.ProductId, oi.Product.Name })
        .Select(g => new
        {
            g.Key.ProductId,
            ProductName = g.Key.Name,
            TotalQuantity = g.Sum(x => x.Quantity),
            TotalRevenue = g.Sum(x => (decimal)x.Quantity * x.UnitPrice)
        })
        .OrderByDescending(x => x.TotalQuantity)
        .Take(TakeCount)
        .ToList();

    var result = data.Select(x => new ProductSalesDto(
        x.ProductId,
        x.ProductName,
        x.TotalQuantity,
        x.TotalRevenue))
        .ToList();

    return Ok(result);
}

```

Synkron variant av "GETTOPPRODUCTS".

```

[HttpGet("top-async")]
1 reference
public async Task<ActionResult<IEnumerable<ProductSalesDto>>> GetTopProductsAsync()
{
    var since = DateTime.UtcNow.AddDays(-DaysWindow);

    var data = await _context.OrderItems
        .Where(oi => oi.Order.OrderDate >= since)
        .GroupBy(oi => new { oi.ProductId, oi.Product.Name })
        .Select(g => new
        {
            g.Key.ProductId,
            ProductName = g.Key.Name,
            TotalQuantity = g.Sum(x => x.Quantity),
            TotalRevenue = g.Sum(x => (decimal)x.Quantity * x.UnitPrice)
        })
        .OrderByDescending(x => x.TotalQuantity)
        .Take(TakeCount)
        .ToListAsync();

    var result = data.Select(x => new ProductSalesDto(
        x.ProductId,
        x.ProductName,
        x.TotalQuantity,
        x.TotalRevenue))
        .ToList();

    return Ok(result);
}

```

Asynkron variant av "GETTOPPRODUCTS".

```

[HttpGet("top-notrack")]
1 reference
public async Task<ActionResult<IEnumerable<ProductSalesDto>>> GetTopProductsNoTrack()
{
    var since = DateTime.UtcNow.AddDays(-DaysWindow);

    var data = await _context.OrderItems
        .Where(oi => oi.Order.OrderDate >= since)
        .GroupBy(oi => new { oi.ProductId, oi.Product.Name })
        .Select(g => new
        {
            g.Key.ProductId,
            ProductName = g.Key.Name,
            TotalQuantity = g.Sum(x => x.Quantity),
            TotalRevenue = g.Sum(x => (decimal)x.Quantity * x.UnitPrice)
        })
        .OrderByDescending(x => x.TotalQuantity)
        .Take(TakeCount)
        .AsNoTracking()
        .ToListAsync();

    var result = data.Select(x => new ProductSalesDto(
        x.ProductId,
        x.ProductName,
        x.TotalQuantity,
        x.TotalRevenue))
        .ToList();

    return Ok(result);
}

```

"GETTOPPRODUCTS" med *AsNoTracking()*.

```

[HttpGet("top-cached")]
1 reference
public async Task<ActionResult<IEnumerable<ProductSalesDto>>> GetTopProductsCached()
{
    const string cacheKey = "topProducts_last100days_top10";

    if (_cache.TryGetValue(cacheKey, out List<ProductSalesDto> cached))
        return Ok(cached);

    var since = DateTime.UtcNow.AddDays(-DaysWindow);

    var data = await _context.OrderItems
        .Where(oi => oi.Order.OrderDate >= since)
        .GroupBy(oi => new { oi.ProductId, oi.Product.Name })
        .Select(g => new
        {
            g.Key.ProductId,
            ProductName = g.Key.Name,
            TotalQuantity = g.Sum(x => x.Quantity),
            TotalRevenue = g.Sum(x => (decimal)x.Quantity * x.UnitPrice)
        })
        .OrderByDescending(x => x.TotalQuantity)
        .Take(TakeCount)
        .AsNoTracking()
        .ToListAsync();

    var result = data.Select(x => new ProductSalesDto(
        x.ProductId,
        x.ProductName,
        x.TotalQuantity,
        x.TotalRevenue))
        .ToList();

    _cache.Set(cacheKey, result, TimeSpan.FromMinutes(5));
    return Ok(result);
}

```

"GETTOPPRODUCTS" med Cache.

```

public BenchmarkService(ProductController productController)
{
    _productController = productController;
}

//Hämta alla produkter
1 reference
public Task<IEnumerable<Product>> GetProductsSync()
{
    var result = _productController.GetProductsSync();
    return Task.FromResult(result.Value ?? Enumerable.Empty<Product>());
}

1 reference
public async Task<IEnumerable<Product>> GetProductsAsync()
{
    var result = await _productController.GetProductsAsync();
    return result.Value ?? Enumerable.Empty<Product>();
}

1 reference
public async Task<IEnumerable<Product>> GetProductsAsNoTracking()
{
    var result = await _productController.GetProductsAsNoTracking();
    return result.Value ?? Enumerable.Empty<Product>();
}

1 reference
public async Task<IEnumerable<Product>> GetProductsCached()
{
    var result = await _productController.GetProductsCached();
    return result.Value ?? Enumerable.Empty<Product>();
}

//Hämta mest sålda produkter
1 reference
public Task<IEnumerable<ProductSalesDto>> GetTopProductsSync()
{
    var result = _productController.GetTopProductsSync();
    return Task.FromResult(result.Value ?? Enumerable.Empty<ProductSalesDto>());
}

1 reference
public async Task<IEnumerable<ProductSalesDto>> GetTopProductsAsync()
{
    var result = await _productController.GetTopProductsAsync();
    return result.Value ?? Enumerable.Empty<ProductSalesDto>();
}

1 reference
public async Task<IEnumerable<ProductSalesDto>> GetTopProductsAsNoTracking()
{
    var result = await _productController.GetTopProductsNoTrack();
    return result.Value ?? Enumerable.Empty<ProductSalesDto>();
}

1 reference
public async Task<IEnumerable<ProductSalesDto>> GetTopProductsCached()
{
    var result = await _productController.GetTopProductsCached();
    return result.Value ?? Enumerable.Empty<ProductSalesDto>();
}

```

BenchmarkService.

```

[Benchmark]
0 references
public async Task GetProductsNoAsync()
{
    await _benchmarkService.GetProductsSync();
}

[Benchmark]
0 references
public async Task GetProductsWithAsync()
{
    await _benchmarkService.GetProductsAsync();
}

[Benchmark]
0 references
public async Task GetProductsAsNoTracking()
{
    await _benchmarkService.GetProductsAsNoTracking();
}

[Benchmark]
0 references
public async Task GetProductsCached()
{
    await _benchmarkService.GetProductsCached();
}

[Benchmark]
0 references
public async Task GetTopProductsNoAsync()
{
    await _benchmarkService.GetTopProductsSync();
}

[Benchmark]
0 references
public async Task GetTopProductsWithAsync()
{
    await _benchmarkService.GetTopProductsAsync();
}

[Benchmark]
0 references
public async Task GetTopProductsAsNoTracking()
{
    await _benchmarkService.GetTopProductsAsNoTracking();
}

[Benchmark]
0 references
public async Task GetTopProductsCached()
{
    await _benchmarkService.GetTopProductsCached();
}

```

Benchmarkklass.